

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

1. IDENTIFICACIÓN DE LA GUÍA

Nombre de la guía:	Taller 3
Código de la guía (No.):	002
Taller(es) o Laboratorio(s) aplicable(s):	
Tiempo de trabajo práctico estimado:	6 horas
Asignatura(s) aplicable(s):	Programación Avanzada
Programa(s) Académico(s) / Facultad(es):	Ingeniería Electrónica / Ingeniería de Telecomunicaciones

COMPETENCIAS	CONTENIDO TEMÁTICO	INDICADOR DE LOGRO
C2: Capacidad para aplicar el diseño de ingeniería para generar soluciones que satisfagan necesidades específicas, considerando la salud, la seguridad y el bienestar públicos, así como factores globales, culturales, sociales, ambientales y económicos C5: Capacidad para reconocer las responsabilidades éticas y profesionales en situaciones de ingeniería y emitir juicios fundamentados, considerando el impacto de las soluciones de ingeniería en contextos globales, económicos, ambientales y sociales	Creación y manipulación de arreglos unidimensionales y multidimensionales con NumPy. Generación de vectores, matrices y datos aleatorios. Operaciones vectorizadas, slicing y cálculos sobre diferentes ejes Representación y análisis de señales y variables físicas mediante arreglos numéricos Visualización de datos con Matplotlib: gráficos de líneas, marcadores, leyendas, rejillas, estilos y subplots. Representación y manipulación de imágenes RGB mediante arreglos tridimensionales, regiones de interés y canales de color. Integración de estructuras de control con arreglos para clasificación de datos según criterios de operación.	Construye y manipula arreglos de NumPy para representar y procesar datos asociados a problemas de ingeniería. Aplica operaciones vectorizadas, slicing y funciones de análisis para obtener información relevante de conjuntos de datos multidimensionales. Modela y analiza señales y variables físicas, identificando valores característicos y condiciones de operación. Construye representaciones gráficas adecuadamente etiquetadas y personalizadas que facilitan la interpretación y comparación de datos. Manipula imágenes digitales mediante operaciones sobre arreglos, identificando regiones de interés y separando sus canales de color. Clasifica e interpreta datos de sensores mediante criterios definidos, identificando estados normales, de alerta o críticos.

2. FUNDAMENTO TEÓRICO

La programación científica constituye una herramienta fundamental en ingeniería, ya que permite modelar, analizar y visualizar fenómenos físicos mediante métodos computacionales. En Ingeniería Electrónica y de Telecomunicaciones, estas herramientas permiten representar señales eléctricas, procesos físicos y mediciones provenientes de sensores mediante arreglos numéricos unidimensionales y multidimensionales.

NumPy proporciona estructuras eficientes para almacenar y procesar estos datos mediante arreglos, permitiendo conocer sus dimensiones, forma y tipo de dato, así como realizar operaciones matemáticas vectorizadas, slicing y cálculos sobre diferentes ejes. Estas características facilitan el procesamiento de grandes conjuntos de datos y reducen la necesidad de utilizar ciclos para determinadas operaciones. Las imágenes digitales también pueden representarse mediante arreglos

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

multidimensionales; por ejemplo, una imagen RGB utiliza dos dimensiones para representar la posición de los píxeles y una tercera dimensión para almacenar los canales rojo, verde y azul.

Complementariamente, Matplotlib permite representar gráficamente la información obtenida mediante gráficos de líneas, marcadores, subplots y otras herramientas de personalización. La adecuada incorporación de títulos, etiquetas, leyendas, rejillas y estilos facilita la interpretación y comparación de señales y datos, permitiendo identificar tendencias, ruido, valores extremos y condiciones de operación en diferentes aplicaciones de ingeniería.

3. OBJETIVO(S)

3.1. Objetivo general

- Aplicar herramientas de programación científica mediante NumPy y Matplotlib para la generación, manipulación, análisis y representación gráfica de datos, señales e imágenes asociadas a problemas de ingeniería electrónica y de telecomunicaciones.

3.2. Objetivos específicos

- Crear y manipular arreglos unidimensionales y multidimensionales con NumPy, utilizando operaciones vectorizadas, generación de datos aleatorios, slicing y funciones de análisis estadístico.
- Representar y analizar señales y variables físicas mediante arreglos numéricos, identificando valores característicos, tendencias y condiciones de operación en aplicaciones de ingeniería.
- Aplicar operaciones sobre matrices y sus diferentes ejes para obtener información como valores promedio, máximos y diferencias entre conjuntos de datos.
- Construir representaciones gráficas utilizando Matplotlib, incorporando títulos, etiquetas, leyendas, marcadores, estilos de línea, rejillas y subplots para facilitar la interpretación de los datos.
- Manipular imágenes digitales como arreglos multidimensionales, utilizando slicing, regiones de interés y separación de canales de color para analizar información visual.
- Integrar estructuras de control de Python con arreglos de NumPy para clasificar y procesar datos de acuerdo con criterios y niveles de operación establecidos en problemas de ingeniería.

4. RECURSOS REQUERIDOS

Los elementos requeridos para realizar el taller son:

- Computador personal con internet.
- Acceso a software tipo Visual Studio Code, Google Colab, Kaggle Notebooks, entre otros.
- Material de estudio entregado en clase (Notebooks).

5. ASPECTOS DE SEGURIDAD

No aplica.

6. PROCEDIMIENTO O METODOLOGÍA PARA EL DESARROLLO

- 6.1 Una señal de voltaje senoidal se define como $v = V \cos(2\pi ft)$, donde V es la amplitud de voltaje y f es la frecuencia de la señal.

- Crear el **arreglo** vector de tiempo t de 0 a 0.5 segundos con 1001 puntos.
- Calcular la función de la respuesta transitoria amortiguada v , para una amplitud de voltaje dada por $V = e^{-10t}$ y una frecuencia 60 Hz.
- Simular una interferencia, generando un arreglo de ruido aleatorio entre -0.2 y 0.2 del mismo tamaño que la señal amortiguada. Suma ambos arreglos para obtener v_{noise} .
- Usando slicing, crear los vectores t_{sub} y v_{sub} que contenga solo uno de cada diez valores de los arreglos con ruido.
- Imprimir el valor máximo y el promedio de la señal con ruido, con dos decimales.
- Crear una figura que cumpla lo siguiente:
 - Tamaño de 10 pulgadas de ancho por 3 pulgadas de alto.
 - Graficar la señal pura v con una línea continua de color negro, sin marcadores.
 - Añadir la señal con ruido v_{noise} , como una línea semitransparente, con un color cyan.
 - Añadir la señal submuestreada con ruido v_{sub} usando únicamente marcadores de color rojo.
 - Agrega título, etiquetas a los ejes ("Tiempo [s]" "Voltaje [V.]"), leyenda y activa la rejilla (grid).

6.2 Análisis de una "Imagen Sintética"

- Generar una matriz de ceros de tamaño 300x300x3 (RGB) usando con tipo de dato uint8.
- Sumarle a la imagen un ruido aleatorio suave en todos los canales para simular interferencia térmica.
- Asignar el valor máximo (255) a los tres canales en un cuadro central (filas 100 a 200, columnas 100 a 200) para simular un "punto caliente".
- Restarle al "punto caliente" un ruido aleatorio suave en todos los canales para simular interferencia térmica.
- Realizar un slicing para extraer solo la **zona cercana** al "punto caliente" (ROI).
- Crear un gráfico compuesto por 4 subfiguras, cada una con su respectivo título.
 - La primera debe ser la imagen sintética original
 - La segunda será la imagen con ruido
 - La tercera será la ROI
 - La cuarta figura debe mostrar solo el canal rojo.

6.3 Un sistema de monitoreo registra la temperatura de dos sensores ubicados en puntos críticos en una caldera durante 60 minutos. Debes generar los datos, analizarlos y generar un reporte visual.

- Generación de datos (**arreglos**):
 - Crear un vector de tiempo t de 0 a 60 minutos con 120 puntos.
 - Generar los datos del Sensor A usando una señal que inicie en 20 y suba gradualmente $0.5t + \text{ruido}$, donde *ruido* corresponde a un arreglo de valores aleatorios entre -3 y 3 °C, del mismo tamaño de t .
 - Generar los datos del Sensor B con valores aleatorios entre 40 y 60 °C.
 - Crea un cuarto arreglo llamado Diferencia que sea la resta punto a punto entre el Sensor A y el Sensor B.
- Crea una figura con dos subplots verticales (2 filas, 1 columna).
 - El primer subplot debe incluir las gráficas de las señales de ambos sensores (A y B) en el mismo eje. Usa colores distintos, añade leyendas ("Sensor A", "Sensor B") y etiquetas a los ejes.
 - El subplot inferior debe tener sólo la señal de Diferencia. Usa un color naranja, un estilo de línea discontinuo ('--') y añade una rejilla (grid) de fondo, y el marcador '1' visto en clase.
 - Añade un título principal a la figura que diga "Reporte de Monitoreo de Caldera".

6.4 Suponga que tiene una matriz de datos donde cada fila representa una hora del día (24 horas) y cada columna un sensor de temperatura diferente (4 sensores en un rack).

- Crea una matriz de 24 x 4 con valores aleatorios entre 15 y 30 grados Celsius.
 - Calcula la temperatura promedio de cada sensor a lo largo del día (usando reducción de dimensiones con *axis*).
 - Identifica cuál fue la temperatura máxima, a qué hora se registró (índice de fila) y el sensor en que ocurrió (índice de columna).
 - Utilice un ciclo **for** para construir un gráfico de cada una de las cuatro señales.
 - Calcule la temperatura promedio en cada hora del día
 - Construya un gráfico de temperatura promedio vs la hora del día.
- Recuerde personalizar las figuras.

6.5 La carga de un capacitor en un circuito RC sigue la fórmula $V_c(t) = V_s(1 - e^{-t/(RC)})$, donde V_s es el voltaje de la fuente, R es la resistencia y C es la capacitancia.

- Grafique la curva de carga para $V_s = 10\text{ V}$, $R = 1\text{ k}\Omega$ y $C = 10\text{ }\mu\text{F}$
- El tiempo debe ir de 0 a 5τ (donde $\tau = RC$). **En el gráfico** el tiempo debe ir en ms (milisegundos).
- La línea debe ser de color azul y estilo discontinuo.
- Agregue la leyenda y la rejilla de fondo (en color rojo y punteada).

6.6 En una planta industrial, un motor reporta su velocidad (RPM) cada segundo. Se han recolectado 101 muestras de la velocidad de un motor. Estos datos se almacenan en una matriz de 101x2, donde la primera columna representa el tiempo (en segundos) y la segunda columna, representa la velocidad (en RPM).

- Crear el vector `time` con los tiempo secuenciales entre 0 y 100 segundos y el vector `velocidad` con los datos de las velocidades aleatorias entre 1000 y 5200 RPM.
- Usa el método `np.column_stack((time, velocidad))` para crear el arreglo de NumPy con dos dimensiones, que incluya los tiempos secuenciales y las velocidades aleatorias.
- Utiliza un ciclo `for` para recorrer el arreglo. Según el valor de la velocidad, clasifica cada punto (el par tiempo-velocidad) en una de tres listas diferentes:
 - Normal:** $Velocidad \leq 4000\text{ RPM}$.
 - Alerta Naranja:** $4000\text{ RPM} < Velocidad \leq 4500\text{ RPM}$.
 - Alerta Roja:** $Velocidad > 4500\text{ RPM}$.
- Grafica los puntos (sin línea) usando marcadores diferentes para cada grupo de datos. Los marcadores de los datos normales en verde, los de alerta naranja en naranja y los críticos en rojo.

6.7 En el mantenimiento preventivo de un centro de datos, se utiliza una cámara térmica que entrega imágenes en formato RGB.

- Importar la imagen de la URL (Pegar el contenido del siguiente cuadro en una única línea)

```
!wget -O "thermal_heatmap.png"
"https://raw.githubusercontent.com/estebangonzalezITM/ProgramacionAvanzada/main/Recursos/Thermal%20Server%20Rack%20Heatmap.png"
```

- Imprimir el tamaño y el tipo de la imagen cargada. Además, imprimir el tipo de elementos que conforman la imagen cargada
- Mediante slicing, aísle una zona específica de un rack donde se observe concentración de calor (región de interés).
- Invierta la imagen (efecto espejo tanto vertical como horizontal) utilizando pasos de slicing negativos. En una misma figura, grafica la imagen sin editar, y después de cada una de las inversiones. Recuerda usar títulos en cada subfigura.
- Manipular los canales de color para resaltar las regiones representadas con mayor intensidad en la imagen térmica.

	GUÍA DE TRABAJO PRÁCTICO - EXPERIMENTAL Talleres y Laboratorios de Docencia ITM	Código	FGL 029
		Versión	03
		Fecha	18-07-2023

- f) Manipule la matriz de la imagen para que únicamente sea visible el canal rojo. Esto permitirá identificar las regiones representadas con mayor intensidad en dicho canal de la imagen térmica.
- g) Utilizando subfiguras, graficar la región de interés original y cada uno de los canales RGB.

7. PARÁMETROS PARA ELABORACIÓN DEL INFORME

No aplica

8. DISPOSICIÓN DE RESIDUOS

No aplica

9. BIBLIOGRAFÍA

- [1] https://github.com/estebangonzalezITM/ProgramacionAvanzada_2026-1
- [2] Langtangen, H. P. (2014). A primer on scientific programming with Python. Springer. ISBN: 978-3-662-49886-6.
- [3] Harris, C.R., Millman, K.J., van der Walt, S.J. et al. (2020). Array programming with NumPy. Nature 585, 357-362. DOI: 10.1038/s41586-020-2649-2.

Elaborado por:	<i>Esteban Gonzalez Valencia</i>
Revisado por:	
Versión:	2
Fecha:	<i>Octubre de 2026</i>